

Jean Victor Borges de Abreu

Análise da Transição para Algoritmos criptográficos Pós-Quânticos nos modelos NIST

Petrópolis -Rio de Janeiro

2026

Jean Victor Borges de Abreu

Análise da Transição para Algoritmos criptográficos Pós-Quânticos nos modelos NIST

Trabalho de Conclusão de Curso apresentado
à

FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

Orientador: Nome do Orientador

Petrópolis -Rio de Janeiro
2026

1 Resumo

2 Abstract

3 Lista de Figuras

Lista de ilustrações

Figura 1 – Diagrama da Simulação do Backdoor do Dual_EC_DRBG	19
Figura 2 – Fluxo da Simulação do Backdoor do Dual_EC_DRBG	20
Figura 3 – Simulação do Ataque de Força Bruta Teste 1	21
Figura 4 – Simulação do Ataque de Força Bruta Teste 2	22

4 Lista de Abreviaturas

ABNT: Associação Brasileira de Normas Técnicas CSPRNG: Cryptographically Secure Pseudo Random Number Generator DRBG: Deterministic Random Bit Generator FAETERJ: Faculdade de Educação Tecnológica do Estado do Rio de Janeiro JEP: JDK Enhancement Proposals KEM: Key Encapsulation Mechanism LCG: Linear Congruential Generator LWE: Learning With Errors ML-KEM: Module-Lattice-Based Key Encapsulation Mechanism NIST: National Institute of Standards and Technology NRBG: Non-deterministic Random Bit Generator PQC: Post-Quantum Cryptography PRNG: Pseudo Random Number Generator QRNG: Quantum Random Number Generator TRNG: True Random Number Generator

Sumário

1	RESUMO	2
2	ABSTRACT	3
3	LISTA DE FIGURAS	4
	Lista de ilustrações	4
4	LISTA DE ABREVIATURAS	5
	Sumário	5
5	INTRODUÇÃO	9
5.1	Contextualização do Tema	9
5.2	Justificativa/Problema de Pesquisa	9
5.3	Motivação/Hipótese e Ameaça Tecnológica	9
5.4	Objetivos	9
5.5	Metodologia	9
6	FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO .	10
6.1	Entropia e Fontes de Incerteza	10
6.2	A Semente (Seed) como Elemento Crítico	11
6.3	Definição de Aleatoriedade e Tipos de Geradores Aleatórios . . .	12
6.4	Avaliação Estatística da Aleatoriedade	13

7	QUEBRA DE CRIPTOGRAFIAS	16
7.1	Visão Geral da Quebra de Criptografias	16
7.2	Falhas Históricas de Aleatoriedade	16
7.2.1	Uma avaliação pratica do Dual_EC_DRBG	17
7.3	Vulnerabilidades em Algoritmos Criptográficos	22
8	REVISÃO DOS PROTOCOLOS E PADRÕES DE CRIPTOGRAFIA	24
8.1	Fundamentação da Autoridade e Tipos de Padrões	24
8.1.1	FIPS	24
8.1.1.1	FIPS 204	24
8.1.1.2	FIPS 205	26
8.1.2	SPs	27
8.1.3	IRs	27
8.1.4	ITL	27
8.1.5	CSD	27
8.2	Principios de Segurança	27
8.2.1	Mérito Técnico e Provas de Segurança	27
8.2.2	Integridade e Influência	27
8.3	Metodologia de Seleção	27
8.3.1	Competições de Algoritmos	27
8.3.2	Fases de Competição	27
8.3.3	Propriedade Intelectual (IP)	27
8.4	Manutenção e Substituição de Padrões	27
8.4.1	Ciclo de Vida e Retirada (Withdrawal)	27
8.4.2	Análise de Constantes e Proveniência	27
9	NIST IR 8105 PLANEJAMENTO PARA A CRIPTOGRAFIA PÓS-QUÂNTICA	28
9.1	Computação Quântica vs. Criptografia Atual	28
9.1.1	Impacto nos Sistemas de Chave Pública (RSA, Diffie-Hellman, ECC)	28
9.1.2	Tabela Comparativa de Impacto	28
9.1.3	Estimativa ded Janela de Ameaça	28
9.2	Famílias de Aalgoritmos Pós-Quânticos	29
9.2.1	Lattice-Based Cryptography	29
9.2.2	Code-Based Cryptography	29
9.2.3	Multivariate Cryptography	30
9.2.4	Hash-Based Cryptography	30
9.2.5	Isogeny-Based Cryptography	30
9.3	Estratégias de Transição e Agilidade Criptográfica	30
9.3.1	Definição de Agilidade Criptográfica (Cripto Agility)	31

9.3.2	Insuficiência do Paradigma Bits-of-security	31
9.4	O Plano de Padronização	32
9.4.1	Processo Fora da Competição Traficional	32
9.4.2	CrITÉrios Iniciais e Prazos	32
10	NIST IR 8547 TRANSIÇÃO PARA A CRIPTOGRAFIA PÓS-QUÂNTICA: ESTRATÉGIAS E CONSIDERAÇÕES	33
10.1	Urgência Prática e o Modelo de Ameaça	33
10.1.1	Ameaça Harvest now, Decrypt Later	33
10.1.2	Teorema de Mosca e a Linha do Tempo	33
10.2	Novos Padrões e Categorias de Segurança	33
10.2.1	Publicações dos primeiros FIPS PQC	34
10.2.2	Categorias de Segurança Pós-Quântica (1 a 5)	34
10.3	Cronograma de Descontinuação (Deprecation)	34
10.3.1	Transição de Assinaturas Digitais (RSA, ECDSA, EdDSA)	34
10.3.2	Transição de Estabelecimento de Chaves (DH, MQV, RSA)	35
10.4	Estratégias de Transição: Protocolos Híbridos	35
10.4.1	Soluções Híbridas (PQC-Classical)	35
10.4.2	Protocolos Híbridos (TLS, SSH, IPsec)	35
10.4.3	Agilidade Criptográfica em Camadas de Infraestrutura	36
11	CRISTALS KYBER	37
11.1	Introdução	37
11.2	Estrutura do CRYSTALS-Kyber	37
12	CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)	38
12.1	Definição e Objetivos da PQC	38
12.2	Paradigmas Matemáticos da PQC	38
12.3	Criptografia Baseada em Reticulados	38
12.4	Learning With Errors (LWE)	38
12.4.1	Implementação Didática em Java (Demonstração)	38
12.5	Padronização Pós-Quântica	38
13	A TRANSIÇÃO PARA PQC NO ECOSISTEMA JAVA	39
13.1	Suporte à Criptografia Pós-Quântica no Java	39
13.2	Crypto-Agility	39
13.2.1	Implementação Prática: Demo de Crypto-Agility	39
13.3	Comparativo de Modelos	39
13.3.1	Implementação Prática: Benchmark Clássico vs. Kyber	39
14	DISCUSSÃO E ANÁLISE CRÍTICA	40

14.1	Impacto em Desempenho	40
14.2	Barreiras de Adoção	40
14.3	Implicações Sociais e Éticas	40
15	CONCLUSÃO	41
15.1	Síntese dos Resultados	41
15.2	Resposta ao Problema de Pesquisa	41
15.3	Trabalhos Futuros	41
	REFERÊNCIAS	42

5 Introdução

A onipresença da criptografia na infraestrutura digital moderna (internet, sistemas bancários, IoT e comunicações seguras).

5.1 Contextualização do Tema

Textos

5.2 Justificativa/Problema de Pesquisa

O conflito entre o determinismo computacional e a necessidade de aleatoriedade em sistemas criptográficos.

5.3 Motivação/Hipotese e Ameaca Tecnológica

O impacto da computação quântica — especialmente por meio do Algoritmo de Shor — sobre os modelos atuais de segurança e pseudoaleatoriedade.

5.4 Objetivos

Objetivo Geral: Analisar a evolução dos geradores pseudoaleatórios criptograficamente seguros frente à transição para a criptografia pós-quântica. Objetivos Específicos: Estudar PRNGs e CSPRNGs clássicos Analisar o impacto dos algoritmos quânticos Investigar os novos paradigmas de pseudoaleatoriedade baseados em reticulados Avaliar a adoção desses conceitos no ecossistema Java

5.5 Metodologia

Pesquisa bibliográfica, análise documental de padrões criptográficos e estudo prático no ecossistema Java.

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO

6.1 Entropia e Fontes de Incerteza

Captação de eventos físicos imprevisíveis como base para sistemas criptográficos seguros.

Entropia é uma medida da aleatoriedade ou imprevisibilidade dos dados. No contexto da criptografia, a entropia é usada para gerar números aleatórios ou chaves que são essenciais para a comunicação segura e a criptografia. Sem uma boa fonte de entropia, os protocolos criptográficos podem se tornar vulneráveis a ataques que exploram a previsibilidade das chaves geradas.

Na maioria dos sistemas operacionais, a entropia é gerada pela coleta de eventos aleatórios de diversas fontes, como interrupções de hardware, movimentos do mouse, pressionamentos de teclas e atividade do disco. Esses eventos são inseridos em um conjunto de entropia, que é então usado para gerar números aleatórios quando necessário.

Protocolos criptográficos, como SSL/TLS, SSH e IPsec, dependem fortemente da entropia para gerar números aleatórios e chaves para criptografia, autenticação e proteção de integridade. A aleatoriedade desses números é crucial para a segurança da comunicação. Se um atacante conseguir prever os números aleatórios usados em um protocolo criptográfico, ele poderá comprometer a confidencialidade, a integridade ou a autenticidade da comunicação. Se a chave mestra prévia for previsível ou tendenciosa, um atacante poderá interceptar a mensagem criptografada, descriptografá-la e obter acesso às chaves de sessão. Isso permitiria ao atacante espionar a comunicação, modificar os dados ou se passar pelo servidor.

Um tipo de entropia. É a entropia de “Shannon”. Que utiliza um tratamento probabilístico de uma função logarítmica. Sendo sua formula:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i) \quad (6.1)$$

Em que:

- $H(X)$ é a entropia em bits;
- p_i é a probabilidade de ocorrência do resultado i ;
- n é o número total de resultados possíveis.

Uma limitação determinística é em cima de um processador de maquina de estados finitos. Se acaso for utilizado as mesmas variáveis, o resultado permanece inalterado. Onde

se um invasor tiver a certeza da seed de origem da criptografia. Ela por si mesma, não é aleatória. Para evitar tais acontecimentos se faz necessário que o software faça a captação da incerteza do mundo físico.

A entropia deve ter força superior ou igual a segurança do sistema criptográfico. Entropia adicional poder ser necessária para compensar a perda de entropia durante o processo de geração de chaves, mas não necessariamente adiona mais segurança. Ter mais entropia do que a quantidade avaliada é aceitável, ter menos do que o recomendado pode ser fatal para a segurança do sistema criptográfico. O comprimento mínimo depende do mecanismo criptográfico específico e do nível de segurança desejado. A entrada de entropia e a seed resultante devem ser consistentes com a segurança dos dados protegidos pelo consumidor. Em um modelo DRBG a seed e as chaves geradas devem possuir o mesmo nível de segurança, pois no modelo DRBG depende da entropia usada, seguindo o critério de segurança crítico (CSP).

6.2 A Semente (Seed) como Elemento Crítico

Falar sobre esse ponto, pois vai ser utilizado na explicação dos protocolos tanto do funcionamento de verificação do NIST 800-22, do NIST IR 8547 e do NIST IR 7977 por quantico

Uma sequência de bits usada como entrada para um gerador de números pseudoaleatórios (DRBG). A semente é a chave inicial, usada internamente para a concepção do DRBG. Sua entropia deve ser suficiente para garantir a segurança do DRBG. A semente é um elemento crítico, pois determina a sequência de números pseudoaleatórios gerados pelo DRBG. Se a semente for previsível ou tiver baixa entropia, o DRBG pode ser vulnerável a ataques que exploram essa fraqueza. Portanto, é essencial que a semente seja gerada a partir de uma fonte de entropia confiável.

Uma sequência de bits usada como entrada para um mecanismo DRBG. A semente determinará uma parte do interior estado do DRBG, e sua entropia deve ser suficiente para apoiar a força de segurança do DRBG Para obter uma seed suficientemente aleatória. Deve se submeter a uma estimativa de entropia para as saídas da fonte de ruído, baseada na própria fonte da entropia. Essa estimativa é denominada de $H_{\text{submitter}}$. Depois de possuir uma trilha de estimativas, a estimativa de entropia mínima por amostra chamada de H_{original} é determinada. Se o tamanho for superior a 256 bits, ele precisará ser reduzido para menor ou igual a 256 bits.

Se o conjunto de dados não for binário, é necessário uma entropia adicional em bit, denominada de $H_{\text{bitstring}}$ é estimada. O conjunto de dados sequências possui L amostras, cada uma contendo N bits e é considerado uma cadeia de bits de tamanho $L \times N$. Após os primeiros 1.000.000 bits podem vir a serem desconsiderados então $H_{\text{bitstring}}$ é calculada. Tendo como amostra estimada como $N \times H_{\text{bitstring}}$. A estimativa da entropia inicial da

fonte é calculada como $H_{\text{initial}} = \min(H_{\text{original}}, H_{\text{bitstring}})$.

6.3 Definição de Aleatoriedade e Tipos de Geradores Aleatórios

RNG (Random Number Generator) é um dispositivo ou algoritmo que gera uma sequência de números ou símbolos que não possuem nenhum padrão discernível, ou seja, são imprevisíveis. Que utiliza uma fonte de incerteza (ou seja, a fonte de entropia) ao mesmo tempo que faz uso de uma função de processo (processo de destilação) para gerar aleatoriedade. O processo de destilação é uma etapa crucial para exceder qualquer ruptura de segurança na origem da entropia que possa resultar em uma sequência não aleatória. Como a geração de longas sucessões de zeros ou uns, ou mesmo a repetição de um mesmo número, o que pode ser um indicativo de uma fonte de entropia comprometida. Normalmente a fonte de entropia é através de um processo físico, como o ruído térmico, circuito elétrico, o movimento do mouse ou a digitação do teclado, ou os efeitos quânticos de um semicondutor. Muitas fontes podem ser utilizadas para esse processo. As saídas de um RNG podem vir a ser utilizadas para um pseudo gerador de números aleatórios (PRNG). A saída de um RNG deve seguir certos critérios de aleatoriedade medidos por testes estatísticos, como os definidos pelo NIST 800-22, para garantir que os números gerados sejam adequadamente aleatórios para uso em criptografia e outras aplicações sensíveis à segurança. Para fins de criptografia, é crucial que os números gerados por um RNG sejam verdadeiramente aleatórios. No entanto, algumas das fontes como data/hora são previsíveis, que combinados com outras fontes de entropia, podem mitigar a previsibilidade. E que ainda pode se tornar deficiente ao passar pelos testes estatísticos. A produção de RNGs de alta qualidade se tornam um recurso oneroso. Tornando improdutivo quando se faz necessário de muitas gerações de RNGs. Por isso, a utilização de PRNGs se torna uma alternativa mais viável, pois eles podem gerar grandes quantidades de números pseudoaleatórios a partir de uma semente inicial.

PRNG (Pseudo-Random Number Generator) é um tipo de gerador de números aleatórios que utiliza um algoritmo determinístico para produzir uma sequência de números que aparenta ser aleatória. Ele faz uso de uma semente (seed) inicial para gerar uma sequência de números pseudoaleatórios. Em momentos que se faz necessário uso de imprevisibilidade, a seed se faz necessária ser imprevisível e oriundas e soma de um RNG. Por padrão um PRNG requer majoritariamente de um RNG. Como o PRNG é um algoritmo determinístico, por isso ele leva o nome de pseudoaleatório, pois a sequência de números gerada por um PRNG é completamente determinada pela semente inicial. Como a reprodução de uma sequência de números pseudoaleatórios é possível se a semente for conhecida, somente ela se faz necessária salvar e proteger. De forma cômica, muitos dos valores gerados por um PRNG aparentam ser mais aleatórios do que os números gerados por um RNG. Se sua concepção for bem desenvolvida, cada geração utiliza o valor da anterior por meio de

transformações matemáticas, que acabam introduzindo uma aleatoriedade adicional. Uma série de transformações podem eliminar autocorrelações e padrões, entre a entrada e a saída.

DRBG (Deterministic Random Bit Generator) é um tipo específico de PRNG que é projetado para ser criptograficamente seguro. Ele produz uma sequência de bits que a partir de uma seed inicial, determina uma sequência de bits pseudoaleatórios. A seed para instanciar um DRBG deve ser gerada a partir de uma fonte de entropia confiável, como um RBG (Random Bit Generator), para garantir a segurança do DRBG. Se a seed for protegida e tiver alta entropia, o DRBG pode ser considerado seguro para uso em criptografia. Além da dependência de uma semente de alta entropia, a segurança de um DRBG reside em sua capacidade de manter a Backtracking Resistance e a Prediction Resistance. Conforme o NIST SP 800-90A, esses mecanismos garantem que o comprometimento temporário do estado interno do gerador não resulte na exposição de chaves criptográficas geradas anteriormente ou no colapso da imprevisibilidade dos bits futuros

NRBG (Non-deterministic Random Bit Generator) é um tipo de gerador de números aleatórios que se baseia em processos físicos ou eventos imprevisíveis para gerar números aleatórios. Ao contrário dos DRBGs, que são determinísticos e dependem de uma semente para gerar números pseudoaleatórios, os NRBGs não são determinísticos e não dependem de uma semente. Eles podem usar fontes de entropia, como ruído térmico, fenômenos quânticos ou outros eventos físicos imprevisíveis para gerar números aleatórios.

TRNG (True Random Number Generator) utiliza da entropia de eventos físicos imprevisíveis para gerar números aleatórios ao invés da utilização de algoritmos determinísticos. É normalmente utilizado para gerar chaves criptográficas, Algoritmo de assinatura digital de curva elíptica[ECDSA] ou chaves MAC simétricas, para armazenar e transmitir dados com alta segurança em protocolos criptográficos como o TLS. As chaves geradas por um TRNG são altamente resistentes a ataques de força bruta, pois a entropia de um TRNG é geralmente muito alta, tornando difícil para um atacante adivinhar ou reproduzir as chaves geradas.

6.4 Avaliação Estatística da Aleatoriedade

Existem diversos testes estatísticos que podem ser utilizados em uma sequência para avaliar e comparar com uma sequência verdadeiramente aleatória. A aleatoriedade é uma característica probabilística, de forma que é possível descrever em termos de probabilidade. O resultado dos avaliadores estatísticos, quando baseados em uma sequência de bits verdadeiramente aleatória, é denominada de priori. É capaz de aplicar uma infinidade de testes estatísticos para avaliar a aleatoriedade de uma sequência de bits. Cada um avaliando um aspecto diferente da aleatoriedade, sendo a presença ou ausência de padrões, que sendo identificados seria uma indicação que a sequência não possui as características

aleatórias suficientes. Já que existe uma possibilidade de aplicar uma sequência infinita de testes, nenhum pode ser considerado completo, sendo necessário estabelecer um conjunto de testes estatísticos padronizados para avaliar a aleatoriedade de sequências de bits. E ainda assim devem ser analisados com cuidado para evitar conclusões errôneas sobre a aleatoriedade de uma sequência de bits.

Uma hipótese nula (H_0) é formulada para cada teste estatístico, que geralmente afirma que a sequência de bits é aleatória. Na outra ponta está a hipótese alternativa (H_1), que afirma que a sequência de bits não é aleatória. O teste estatístico é então aplicado à sequência de bits, e um valor de teste é calculado com base nos resultados do teste. Ao final de cada teste se tem uma decisão a ser tomada, que é se rejeitar ou não a hipótese nula.

Para cada distribuição teórica em cima da hipótese nula (H_0) é determinada por meios matemáticos. A partir disso, na distribuição de referência, um valor crítico é determinado longe o suficiente para que a probabilidade de obter um resultado tão extremo quanto o observado seja menor do que um nível de significância predefinido 0.01.

Na realização, se um valor de um teste exceder o valor crítico, a hipótese nula (H_0) é rejeitada. Se o pressuposto da aleatoriedade for verdadeira o valor final da estatística terá uma probabilidade muito baixa de exceder o valor crítico.

Só há duas alternativas de decisão, ou a hipótese nula (H_0) é rejeitada ou não é rejeitada. Se a hipótese nula for rejeitada, isso sugere que a sequência de bits não é aleatória, e que pode haver padrões ou correlações presentes na sequência. Se a hipótese nula (H_0) não for rejeitada, isso sugere que a sequência de bits é aleatória, mas não prova que a sequência é verdadeiramente aleatória.

Erro Tipo 1 ocorre quando a hipótese nula (H_0) é rejeitada, mesmo que seja verdadeira. Isso ocorre quando o teste estatístico indica que a sequência de bits não é aleatória. A probabilidade de um erro do tipo 1 é chamada de nível de significância e é denotada por α . Na criptografia valores comuns para α são 0.01 ou 0.001, o que significa que há uma probabilidade de 1% ou 0.1% de rejeitar a hipótese nula (H_0) quando ela é verdadeira. É esperado que 1 a cada 100 sequências de bits sejam rejeitadas.

Erro do Tipo 2 ocorre quando a hipótese nula (H_0) não é rejeitada, mesmo que seja falsa. Isso ocorre quando o teste estatístico indica que a sequência de bits é aleatória, mesmo que haja padrões ou correlações presentes na sequência. A probabilidade de um erro do tipo 2 é denotada por β . Não há um valor fixo para β , pois depende do teste estatístico específico e da natureza dos padrões ou correlações presentes na sequência de bits e que possui uma infinidade de maneiras de que uma sequência de bits não seja aleatória.

Uma das principais metas da execução de testes estatísticos é minimizar a probabilidade de cometer erros do tipo 2. Diminuindo a probabilidade de ter um gerador ruim aceitar uma sequência de bits como aleatória. O estatístico de teste é usado para calcular um

P-value que resume a força da evidência contra a hipótese nula.

Se $P\text{-value} = 1$, a sequência parece ter aleatoriedade perfeita.

Se $P\text{-value} = 0$, a sequência parece ser completamente não-aleatória.

Se $P\text{-value} \geq \alpha$, a hipótese nula é aceita (a sequência parece ser aleatória).

Se $P\text{-value} < \alpha$, a hipótese nula é rejeitada (a sequência não é aleatória).

O NIST SP 800–22 é um conjunto de testes estatísticos amplamente utilizado para avaliar a aleatoriedade de sequências de bits geradas por geradores de números aleatórios. Sendo ao todo 15 testes, cada um projetado para detectar diferentes tipos de padrões ou falhas na aleatoriedade. Esses testes incluem:

1. Teste de Frequência (monobit)
2. Teste de Frequência dentro de um bloco
3. Teste de Sequências
4. Testes para Maior Sequência de Uns em um bloco
5. Testes de Posto de Matriz Binária
6. Teste da Transformada Discreta de Fourier (Espectral)
7. Teste de Correspondência de Modelos não Sobrepostos
8. Teste de Correspondência de Modelos Sobrepostos
9. Teste "Estatístico Universal" de Maurer
10. Teste de Complexidade Linear
11. Teste Serial
12. Teste de Entropia Aproximada
13. Teste de Somas Cumulativas (Cusums)
14. Teste de Excursões Aleatórias
15. Teste de Variante de Excursões Aleatórias

7 Quebra de criptografias

7.1 Visão Geral da Quebra de Criptografias

A quebra de criptografia refere-se ao processo de comprometer a segurança de um sistema criptográfico, seja por meio da descoberta de vulnerabilidades, exploração de falhas ou desenvolvimento de técnicas que permitem a recuperação de informações protegidas sem autorização. A quebra pode ocorrer devido a falhas na implementação, fraquezas nos algoritmos ou avanços tecnológicos que tornam os métodos de ataque mais eficazes. A evolução da computação, especialmente com o advento da computação quântica, tem levantado preocupações significativas sobre a segurança dos sistemas criptográficos atuais, destacando a necessidade de desenvolver e adotar algoritmos resistentes a ataques futuros.

7.2 Falhas Históricas de Aleatoriedade

Dual_EC_DRBG, um gerador de números pseudoaleatórios que foi suspeito de conter uma backdoor, permitindo que um atacante previsse os números gerados.

DUAL_EC_DRBG (Dual Elliptic Curve Deterministic Random Bit Generator) é um fornecedor de dados pseudoaleatórios para iniciar e continuar a operação criptográfica. Seu uso começou em meados dos anos 2000. Ele utiliza duas curvas elípticas, P e Q , para gerar números pseudoaleatório, que são difíceis de prever o próximo valor ou os anteriores.

Curvas elípticas são uma curva algébrica definida por uma equação do tipo $y^2 = x^3 + ax + b$, onde a e b são constantes que satisfazem a condição de que a curva de não singularidade $4a^3 + 27b^2 \neq 0$. Elas são usadas em criptografia devido à sua estrutura matemática que permite operações eficientes e seguras, como a multiplicação de pontos, que é a base para muitos algoritmos criptográficos.

Utilizada em dispositivos moveis ou servidores que lidam com muitas conexões. Sendo corretamente implementada, é considerada segura quanto outras formas de criptografia. Ela é capaz de combinar de melhor eficiencia com segurança permitindo chaves menores do que as usadas em RSA, por exemplo, para um nível de segurança equivalente. Porém o Dual_EC_DRBG foi amplamente criticado por sua lentidão.

O Dual_EC_DRBG utiliza dois pontos, P e Q , em uma curva elíptica. Os valores pseudoaleatórios são gerados a partir da aritmética desses pontos; especificamente, ao utilizar o ponto P , é possível encontrar o seu dobro, $2P$, através da intersecção da reta tangente com a curva e posterior reflexão em relação ao eixo x . Esse processo de multiplicação escalar é perpetuado até atingir um ponto nP .

O algoritmo funciona utilizando um dado inicial (semente) s_i . O fluxo de geração segue os seguintes passos:

- **Atualização de Estado:** O novo estado interno é gerado ao computar o ponto $s_i P$, onde o valor da coordenada x resultante é definido como $s_{i+1} = x(s_i P)$.
- **Geração de Saída:** Simultaneamente, a saída é gerada através do ponto Q no processo $r_i = x(s_i Q)$.

Por padrão, são descartados os 16 bits mais significativos de r_i para compor a saída final utilizada na criptografia. Em termos de fluxo, o valor $s_i P$ impõe-se como o valor inicial para o próximo ciclo, enquanto $x(s_i Q)$ é o valor de saída. Para a segurança do gerador, não deve ser possível reverter s_i a partir da saída, nem que $s_i Q$ revele informações sobre o estado interno $s_i P$.

A vulnerabilidade do **Dual_EC_DRBG** reside justamente nos 16 bits truncados, que reduzem o espaço de busca para apenas $2^{16} = 65536$ possibilidades, tornando viável um ataque de força bruta para recuperar o ponto original. Caso exista uma relação especial entre os pontos tal que $P = zQ$, um atacante que conheça o escalar z pode deduzir, por meio de propriedades associativas, a seguinte igualdade:

$$z(s_i Q) = s_i(zQ) = s_i P$$

7.2.1 Uma avaliação pratica do Dual_EC_DRBG

Para a validação prática da vulnerabilidade, foi desenvolvida uma Prova de Conceito (PoC) em linguagem Java. A implementação utilizou a biblioteca Bouncy Castle para operações de criptografia de curva elíptica. O código fonte seguiu a estrutura lógica de implementações de referência disponíveis na comunidade de segurança sendo adaptado para demonstrar especificamente a previsibilidade de estados no ambiente Java 21.

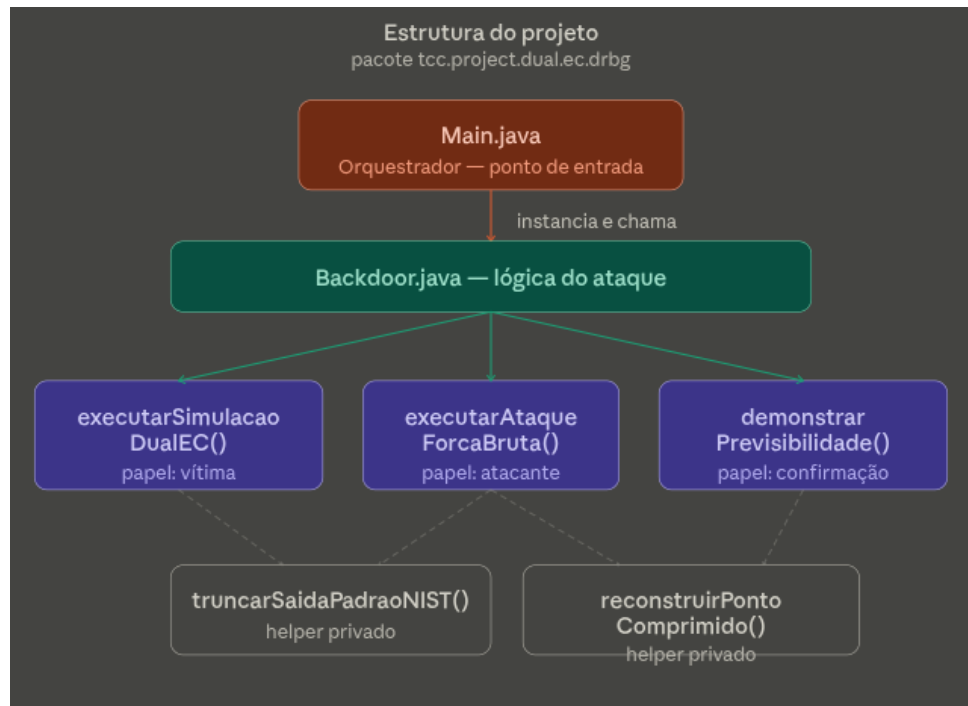
Tabela 1 – Dicionário de Variáveis da Classe Main

Variável	Tipo	Descrição Acadêmica
curve	X9ECParameters	Contém os parâmetros estruturais da curva elíptica NIST P-256.
P	ECPoint	Ponto gerador base da curva, utilizado para a transição de estados internos (s_{i+1}).
n	BigInteger	A ordem da curva elíptica (número total de pontos no corpo finito).
d	BigInteger	O <i>Backdoor</i> . Escalar secreto tal que $Q = d \cdot P$.
Q	ECPoint	Ponto público viciado inserido no gerador para permitir a criptoanálise.
e	BigInteger	Inverso multiplicativo de d em relação a n ($d^{-1} \pmod{n}$).
s	BigInteger	Semente (<i>seed</i>) secreta inicial que define a entropia do sistema.
r0Capturado	byte[]	Primeira amostra de saída (240 bits) observada pelo atacante.
s1Recuperado	BigInteger	O segredo interno descoberto após a execução do ataque de força bruta.

Tabela 2 – Dicionário de Métodos e Variáveis da Classe Backdoor

Identificador	Tipo	Descrição Técnica
executarSimulacao	Método	Implementa as funções de transição de estado e saída do Dual_EC_DRBG.
executarAtaque	Método	Algoritmo de busca exaustiva para recuperação de estado via <i>backdoor</i> .
s_state	BigInteger	Representa o estado interno atual (s_i) no ciclo do gerador.
totalTentativas	int	Contador do esforço computacional (máximo de 2^{17} operações).
xCandidate	BigInteger	Coordenada x reconstruída combinando bits adivinhados e bits observados.
compressionPrefix	String	Prefixo (02 ou 03) que resolve a ambiguidade da coordenada y na curva.
s1_candidate	BigInteger	Candidato a estado interno obtido pela aplicação do escalar e sobre o ponto.
truncarSaida	Método	Simula o descarte dos 16 bits mais significativos da coordenada x .
reconstruirPonto	Método	Formata a coordenada x para o padrão de decodificação da Bouncy Castle.

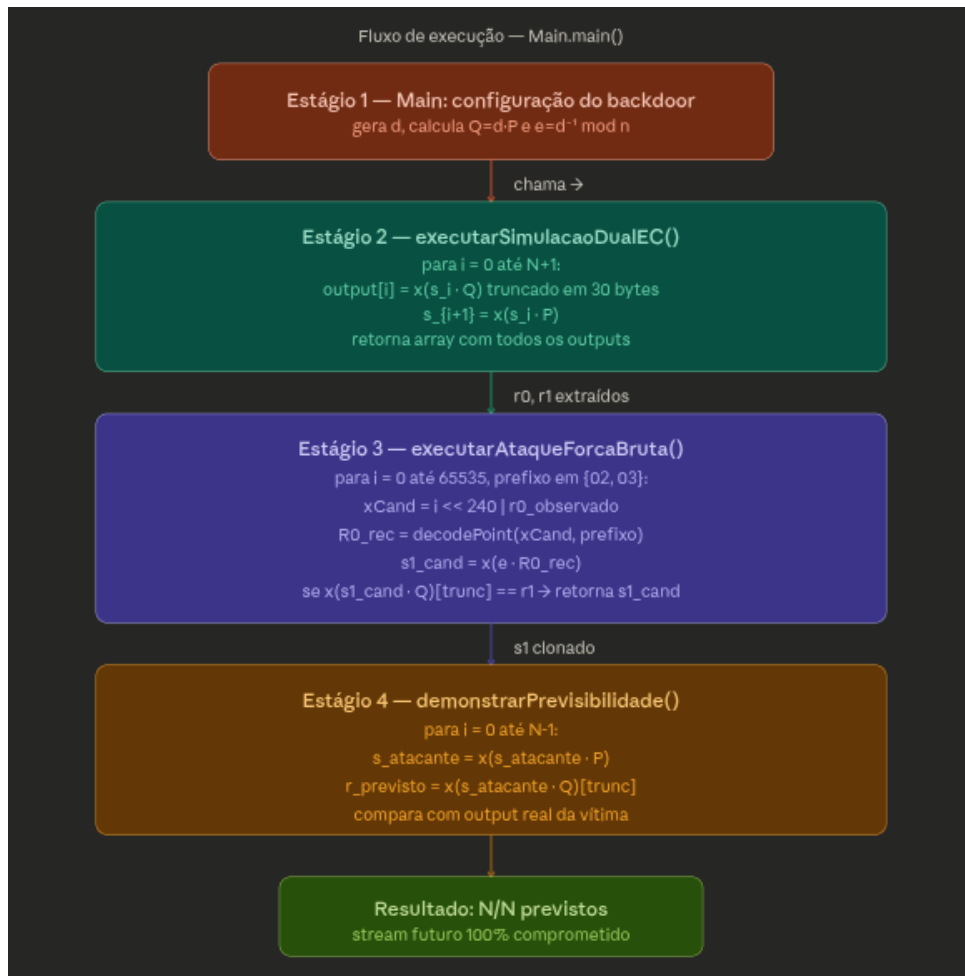
Figura 1 – Diagrama da Simulação do Backdoor do Dual_EC_DRBG



Fonte: Elaborado pelo autor (2026)

Diagrama é uma representação visual do processo de simulação do backdoor do Dual_EC_DRBG. Ele ilustra as etapas envolvidas na geração de números pseudoaleatórios e na execução do ataque de força bruta para recuperar o estado interno. Esse processo foi dividido entre a Main como orquestrador e a classe Backdoor, que contém a lógica de simulação e ataque, além de funções auxiliares. **Main:** Configura os parâmetros da curva elíptica, pontos P e Q, semente inicial e chama os métodos de simulação e ataque. **Backdoor:** Contém o método `executarSimulacaoDualEC` para simular a geração de números pseudoaleatórios e o método `executarAtaqueForcaBruta` para recuperar o estado interno a partir das saídas observadas. Ele também inclui funções auxiliares para reconstruir pontos e truncar saídas conforme o padrão NIST.

Figura 2 – Fluxo da Simulação do Backdoor do Dual_EC_DRBG



Fonte: Elaborado pelo autor (2026)

Um fluxo de execução é a ordem exata em que as instruções de um programa são processadas pelo computador. Diferente do fluxo original de funcionamento do DUAL_EC_DRBG, e como atacantes utilizam sua fragilidade. Na simulação todas as etapas foram implementadas e gerenciadas ao mesmo tempo, se conversando. Na realidade, essas funções e etapas não se conversam. O ataque começa com R_0 e R_1 (os dois outputs observados). O atacante não olha para R_2 durante o processo de quebra — ele prevê R_2 de forma independente, e só ao final compara com o valor real para confirmar que acertou. Isso é idêntico ao que aconteceria na vida real: o servidor TLS emite R_0 num handshake, R_1 num pacote seguinte, e R_2 em outro pacote posterior — todos trafegam pela rede, e o atacante os captura em sequência. R_2 já chegou pela rede antes mesmo de o atacante terminar o brute-force dos 16 bits.

Figura 3 – Simulação do Ataque de Força Bruta Teste 1

```

Projeto > QuebraCripto > src > main > java > tcc > project > dual > ec > drbg > - : Main.java > ...
0
You: 5 minutos ago | 1 author (You)
10 public class Main {
11     Run [Debug]
12     public static void main(String[] args) {
13         Backdoor backdoor = new Backdoor();
14         XECParameters curve = NISTNamedCurves.getByName("P-256");
15         ECPPoint P = curve.getG();
16         BigInteger n = curve.getN();
17
18         BigInteger d = new BigInteger(256, new SecureRandom()).mod(n);
19         ECPPoint Q = P.multiply(d).normalize();
20         BigInteger e = d.modInverse(n);
21
22         System.out.println("=== SIMULACAO DUAL_EC_DRBG - PREVISAO DE GERACAO ===\n");
23
24         final int OUTPUTS_FUTUROS = 10;
25         BigInteger s = new BigInteger(256, new SecureRandom()).mod(n);
26
27         BigInteger[] estadosReais = new BigInteger(OUTPUTS_FUTUROS + 2);
28
29         byte[] outputResultado = backdoor.executarSimulacaoDualEC(OUTPUTS_FUTUROS, curve, P, Q, s, estadosReais);
30
31         byte[] rRCapturado = outputResultado(0);
32         byte[] rICapturado = outputResultado(1);
33
34         System.out.println("... OUTPUTS OBSERVADOS PELO ATACANTE ...");
35         System.out.println("R0 (capturado): " + Hex.toHexString(rRCapturado));
36         System.out.println("R1 (capturado): " + Hex.toHexString(rICapturado));
37         System.out.println();
38
39         BigInteger sRecuperado = backdoor.executarAtaqueForçaBruta(rRCapturado, rICapturado, curve, Q, e);
40
41         backdoor.demonstrarPrevisibilidade(OUTPUTS_FUTUROS, sRecuperado, P, Q, outputResultado);
42     }
43 }
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Fonte: Elaborado pelo autor (2026)

A comprovação prática da vulnerabilidade do Dual_EC_DRBG durante essa simulação demonstra de forma clara que após o atacante ter acesso aos dois primeiros outputs (R0 e R1), ele pode realizar um ataque de força bruta para recuperar o estado interno do gerador. E com isso ele consegue não só prever o R2, mas também todos os outputs futuros, comprometendo completamente a segurança do sistema que depende desse gerador para suas operações criptográficas.

Figura 4 – Simulação do Ataque de Força Bruta Teste 2

```

10 public class Main {
11     public static void main(String[] args) {
12         Backdoor backdoor = new Backdoor();
13         XECParameters curve = NISTNamedCurves.getByName("P-256");
14         ECPublicKey P = curve.getG();
15         BigInteger n = curve.getN();
16         BigInteger d = new BigInteger(256, new SecureRandom()).mod(n);
17         ECPublicKey Q = P.multiply(d).normalize();
18         BigInteger e = d.modInverse(n);
19         System.out.println("=== SIMULACAO DUAL_EC_DRBG - PREVISAO DE GERACAO ===\n");
20         final int OUTPUTS_FUTUROS = 10;
21         BigInteger s = new BigInteger(256, new SecureRandom()).mod(n);
22         BigInteger[] estadosReais = new BigInteger(OUTPUTS_FUTUROS + 2);
23         byte[][] outputResultado = backdoor.executarSimulacaoDualEC(OUTPUTS_FUTUROS, curve, P, Q, s, estadosReais);
24         byte[] rCapturado = outputResultado[0];
25         byte[] rICapturado = outputResultado[1];
26         System.out.println("--- OUTPUTS OBSERVADOS PELO ATACANTE ---");
27         System.out.println("R0 (capturado): " + Hex.toHexString(rCapturado));
28         System.out.println("R1 (capturado): " + Hex.toHexString(rICapturado));
29         System.out.println();
30         BigInteger sRecuperado = backdoor.executarAtaqueForçaBruta(rCapturado, rICapturado, curve, Q, e);
31         backdoor.demonstrarPrevisibilidade(OUTPUTS_FUTUROS, sRecuperado, P, Q, outputResultado);
32     }
33 }

```

```

--- OUTPUTS OBSERVADOS PELO ATACANTE ---
R0 (capturado): c98bed9d573a9d53f629d1b52bed733deb78e48c2ae818bdfbc838df010
R1 (capturado): 3d4c7021972dc64b2e1818c2d94569377a8cf67c9597912bad8c7950c2b

--- EXECUTANDO CRIPTOMALISE (Brute-force 16-bits) ---
[!] Estado interno s1 sincronizado com sucesso!
[!] Esforço computacional: 104.265 tentativas.

--- PREDICÇÃO DE FLUXO DE DADOS (STREAM PREDICTION) ---
Output Valor Previsto pelo Atacante Status
R2 3341bcd7188dc1458e378bf56ff15c7ae415f79bdc2012b05d471fe79b1 [CONFIRMADO]
R3 91608b1354ad6a9f526a27e2ac23223a9d9d949221726d4df623080e [CONFIRMADO]
R4 67ece1669e891da8321e50e74d267495fc57b753d7d5430247f0e5719a8c [CONFIRMADO]
R5 67b77a14a72db70a877fcf435abc767867c0b87ec48befd775ce7d0d6e6f6 [CONFIRMADO]
R6 78ba35c4f618b6a27921644de01987b28f7f61a99efaf576cd015073a2 [CONFIRMADO]
R7 16d901c58ed435afe5daccb700094ba4c192eaa79b8bdab6f11d272aa [CONFIRMADO]
R8 fccba8665f514e492db16fe929b7348208e95c79c0085db5c3195d4d3 [CONFIRMADO]
R9 c34f4e07c54cf6ba8b32b4672f6e74ae214682b6e4e0ba31e4b3960f1f [CONFIRMADO]
R10 90b849e2be50808f38c82733b5a19982c6f6e2043709314580e441120a0 [CONFIRMADO]
R11 78d628f7e1ebf5993dc4e61c6ee0b7b36ce6124448bc61fd539a3a4b7fc [CONFIRMADO]

Resultado da Predição: 10/10 acertos.

```

Fonte: Elaborado pelo autor (2026)

Apesar de o truncamento especificado pelo padrão NIST SP 800-90A remover apenas 16 bits da coordenada x , o custo efetivo do ataque de recuperação de estado é da ordem de 2^{17} multiplicações escalares. Isso ocorre porque, para cada candidato reconstruído de x , é necessário considerar ambas as possíveis coordenadas y , correspondentes às soluções par e ímpar da equação da curva elíptica. Formalmente, o custo total pode ser expresso como:

$$2^{16} \times 2 = 2^{17}$$

No experimento realizado, observou-se que a sincronização do estado exigiu 104.265 tentativas, valor consistente com o limite teórico máximo de 131.072 operações.

O Bug do Debian OpenSSL (2008): Um desenvolvedor removeu uma linha de código que gerava entropia porque ela causava avisos no depurador. O resultado foi que apenas 32.768 chaves SSH eram possíveis. Um ataque de força bruta levava segundos.

Ataques em Cartões Inteligentes (Taiwan): Milhares de IDs digitais foram gerados com sementes fracas, permitindo que chaves privadas RSA fossem fatoradas a partir das chaves públicas.

7.3 Vulnerabilidades em Algoritmos Criptográficos

Algoritmo de grover. capaz de reduzir a força das entropias das seed pela metade: <https://learn.microsoft.com/es-es/azure/quantum/concepts-grovers> Algoritmo de Grover

e a Semente: Explique que o ataque de Grover não "quebra" a matemática do AES ou da Seed, mas reduz a busca exaustiva. Se você usa uma semente de 128 bits, para um computador quântico ela tem o esforço de apenas 2^{64} . Por isso, o NIST IR 7977 exige o dobro de bits (256). Algoritmo de Shor: Este é o "assassino" do RSA e da Curva Elíptica. Enquanto o Grover reduz a força, o Shor aniquila o RSA completamente. É a razão principal da existência da Criptografia Pós-Quântica (PQC).

Colisões em SHA-1: Cite o projeto SHAttered (Google), que provou na prática que o SHA-1 é inseguro ao gerar dois arquivos PDF diferentes com o mesmo hash.

8 Revisão dos protocolos e padrões de criptografia

O NIST utiliza varios documentos para estabelecer os protocolos e padrões de criptografia, cada um com um propósito específico. Sendo comummente publicados em três categorias principais: FIPS (Federal Information Processing Standards), SPs (Special Publications) e IRs (Interagency Reports). Esses documentos são desenvolvidos para garantir a segurança e a integridade dos sistemas de informação, fornecendo diretrizes claras para a implementação de algoritmos criptográficos e práticas de segurança.

8.1 Fundamentação da Autoridade e Tipos de Padrões

8.1.1 FIPS

Os documentos FIPS são padrões de processamento de informações federais que estabelecem requisitos específicos para algoritmos criptográficos e práticas de segurança. Eles são desenvolvidos para garantir a interoperabilidade e a segurança dos sistemas de informação utilizados pelo governo dos Estados Unidos. Os FIPS são amplamente adotados por organizações governamentais e privadas como referência para a implementação de criptografia segura. Esses padrões abrangem uma variedade de tópicos, incluindo algoritmos de criptografia simétrica e assimétrica, funções de hash, e protocolos de segurança.

8.1.1.1 FIPS 204

ML-DSA (Module-Lattice-Based Digital Signature Algorithm) é um padrão para assinaturas digitais pós-quânticas. As assinaturas digitais são utilizadas na identificação e autenticação de documentos eletrônicos. Ele se baseia no CRYSTALS-Dilithium, onde se faz uso de problemas reticulados modulares. (Module-LWE e Module-SIS). Para cada nível de segurança o ML-DSA oferece níveis de proteção diferentes. Sendo:

- ML-DSA-44: Nível de Segurança de 128 bits para recursos limitados
- ML-DSA-64: Nível de Segurança de 192 bits recomendado para a maioria dos casos
- ML-DSA-84: Nível de Segurança de 256 bits longo prazo e alta segurança

O ML-DSA é considerado suficientemente seguro e fortemente infalsificável, mesmo frente a ataques de computadores quânticos. Podendo ser usado para identificar modificações sem autorização, garantindo a integridade e autenticidade dos dados. Além de demonstrar a propriedade de não repúdio, onde o remetente não pode negar a autoria

da assinatura digital. Ele foi desenvolvida de forma que mesmo que o atacante conheça ou possua parte da chave privada ou a própria assinatura, ele não consiga forjar uma assinatura adicional ou modificar a assinatura existente sem ser detectado.

A segurança se dá aos problemas articulados com erros LWE (Learning With Errors) e SIS (Short Integer Solution), que são considerados difíceis de resolver mesmo para computadores quânticos. O problema LWE consiste em recuperar um vetor $\mathbf{s} \in \mathbb{Z}_q^n$ dado um conjunto de equações lineares ruidosas da forma:

$$\mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{b}$$

onde $\mathbf{A}$ e $\mathbf{b}$ são dados e $\mathbf{e}$ é um vetor de erro pequeno. O problema SIS, por sua vez, envolve encontrar um vetor inteiro curto $\mathbf{x}$ tal que:

$$\mathbf{A}\mathbf{x} \equiv \mathbf{0} \pmod{q}$$

onde $\mathbf{A}$ é uma matriz dada e q é um módulo. Ambos os problemas são considerados difíceis de resolver, mesmo para computadores quânticos, o que torna o ML-DSA uma escolha segura para assinaturas digitais pós-quânticas.

A transição da versão 3.1 do CRYSTALS-Dilithium para o padrão final **ML-DSA** (FIPS 204) focou no fortalecimento da segurança contra ataques de colisão e na robustez da implementação. As principais mudanças envolvem:

- **Ajuste de Comprimento de Bits:** Para evitar que ataques de colisão no SHAKE256 degradassem o nível de segurança do NIST (especificamente da categoria 5 para a 4), o comprimento do representante da mensagem μ foi aumentado de **384** para **512** bits. Da mesma forma, a semente aleatória privada ρ' foi expandida de **384** para **512** bits.
- **Fortalecimento de Variáveis de Hash:** O hash da chave pública tr foi padronizado em **512** bits. Além disso, o hash do compromisso $\tilde{c}$ agora utiliza **384** bits para o conjunto *ML-DSA-65* e **512** bits para o *ML-DSA-87*, sendo que todos os bits de $\tilde{c}$ são agora utilizados na geração do desafio c .
- **Segurança de Assinatura e Aleatoriedade:** O ML-DSA introduziu uma variante de assinatura “protegida”, onde o valor π' é derivado de forma pseudorandômica utilizando a chave privada, a mensagem e uma string de aleatoriedade **rnd** de **256** bits. Isso mitiga riscos associados ao determinismo puro em ambientes vulneráveis a ataques de canal lateral.
- **Correções de Integridade e Domínio:** Foi restaurada a verificação de entradas malformadas no desempacotamento da dica (*hint*), garantindo que o esquema seja fortemente infalsificável (sEUF-CMA). Para evitar o uso indevido de chaves, foi implementada a separação de domínio tanto na geração de chaves quanto nas

assinaturas, distinguindo mensagens assinadas diretamente das versões pré-hash (*HashML-DSA*).

8.1.1.2 FIPS 205

SLH-DSA (Signature-Lattice-Based Digital Signature Algorithm) é um padrão para assinaturas digitais sem estado baseado em hash. O SLH-DSA é baseado no SPHINCS+ (Stateless Practical Hash-based Incredibly Nice Collision-resistant Signatures) uma assinatura digital pos-quântico a base de hash, que gera uma assinatura digital com uma chave privada, a verificação ocorre através da chave pública correspondente, não sendo iguais. Cada signatário possui um par de chaves privada e pública. As chaves públicas são de conhecimento público, sendo utilizada justamente para a verificação da assinatura.

A dificuldade do SLH-DSA está na dificuldade de encontrar pre-conditions no hash utilizado e suas várias propriedades relacionais ao hash. Ele foi desenvolvido para suportar ataques em larga escala, mesmo sendo de origem quântica.

O SLH-DSA utiliza de outros dois esquemas de assinatura sendo eles o FORS (Forest of Random Trees) e o XMSS (Xtended Merkle Signature Scheme) o XMSS é um esquema de assinatura de uso único baseada em hash (WOTS+) Winternitz One-Time Signature Plus.

O SLH-DSA é constituído de um grupo de chaves em pares FORMS, no padrão adotado o par de chaves é 2^{63} que são gerados de forma pseudoaleatória por uma seed inicial. O FORMS assina cada par de chaves uma parte do número de mensagens.

XMSS é uma assinatura multitemporária usando uma combinação de assinaturas WOTS+ para cada par de chaves, além de uma assinatura XMSS para o root da árvore. A raiz da árvore é gerada por meio de uma função de hash aplicada a um conjunto de chaves públicas.

A segurança “sem estado” é alcançada através de uma hiperárvore composta por d camadas de árvores XMSS:

- **Camada Superior ($d - 1$):** Consiste em uma única árvore XMSS cuja raiz é a chave pública do sistema ($PK.root$).
- **Camadas Intermediárias:** Cada árvore XMSS assina a raiz da árvore XMSS situada na camada imediatamente inferior.
- **Camada Inferior (Camada 0):** As chaves WOTS+ desta camada são utilizadas para assinar e autenticar as chaves públicas FORMS.

Para assinar uma mensagem, o algoritmo executa os seguintes passos:

1. Calcula um *hash* da mensagem para selecionar uma chave FORS de forma pseudoaleatória.

2. Gera a assinatura FORS para a mensagem.
3. Cria uma assinatura de hiperárvore, que consiste na sequência de assinaturas XMSS necessárias para ligar a chave FORS utilizada até a raiz pública (***PK.root***).

A assinatura final é a combinação da assinatura FORS com esse caminho de autenticação (assinatura de hiperárvore).

- **Chave Pública:** Composta pela raiz da hiperárvore (***PK.root***) e uma semente (***PK.seed***) para separação de domínio.
- **Chave Privada:** Inclui sementes (***SK.seed*** e ***SK.pr f***) que permitem regenerar todos os valores secretos e gerar *hashes* aleatórios, garantindo que o sistema seja determinístico e dispense o armazenamento de estado sobre quais chaves já foram utilizadas.

8.1.2 SPs

8.1.3 IRs

8.1.4 ITL

8.1.5 CSD

8.2 Princípios de Segurança

8.2.1 Mérito Técnico e Provas de Segurança

8.2.2 Integridade e Influência

8.3 Metodologia de Seleção

8.3.1 Competições de Algoritmos

8.3.2 Fases de Competição

8.3.3 Propriedade Intelectual (IP)

8.4 Manutenção e Substituição de Padrões

8.4.1 Ciclo de Vida e Retirada (Withdrawal)

8.4.2 Análise de Constantes e Proveniência

9 NIST IR 8105 Planejamento para a Criptografia Pós-Quântica

9.1 Computação Quântica vs. Criptografia Atual

A computação quântica é um campo emergente que tem o potencial de revolucionar a forma como processamos informações. No entanto, ela também apresenta desafios significativos para a segurança da informação, especialmente no que diz respeito à criptografia. A criptografia atual, baseada em algoritmos como RSA e ECC, é vulnerável a ataques de computadores quânticos, que podem quebrar esses sistemas de segurança em um tempo relativamente curto. A NIST (National Institute of Standards and Technology) reconhece essa ameaça e está trabalhando para desenvolver novos algoritmos de criptografia que sejam resistentes a ataques quânticos. Esses algoritmos, conhecidos como criptografia pós-quântica, são projetados para garantir a segurança das informações mesmo na era da computação quântica. A transição para a criptografia pós-quântica é crucial para proteger dados sensíveis e garantir a segurança das comunicações no futuro.

9.1.1 Impacto nos Sistemas de Chave Pública (RSA, Diffie-Hellman, ECC)

Os sistemas de chave pública, como RSA, Diffie-Hellman e ECC, são amplamente utilizados para garantir a segurança das comunicações e a proteção de dados. No entanto, esses sistemas são vulneráveis a ataques de computadores quânticos devido à sua dependência de problemas matemáticos que podem ser resolvidos eficientemente por algoritmos quânticos, como o algoritmo de Shor. O algoritmo de Shor pode fatorar grandes números inteiros e calcular logaritmos discretos em tempo polinomial, o que torna os sistemas de chave pública atuais inseguros contra ataques quânticos. Isso significa que, com o avanço da computação quântica, os dados protegidos por esses sistemas podem ser facilmente comprometidos, destacando a necessidade urgente de desenvolver e adotar algoritmos de criptografia pós-quântica para garantir a segurança das informações no futuro.

9.1.2 Tabela Comparativa de Impacto

9.1.3 Estimativa da Janela de Ameaça

A janela de ameaça para a quebra dos sistemas de chave pública atuais devido à computação quântica é estimada em cerca de 10 a 20 anos, dependendo do ritmo de desenvolvimento da tecnologia quântica. Durante esse período, é crucial que as organizações e governos comecem a planejar a transição para algoritmos de criptografia pós-quântica.

Algoritmo	Vulnerabilidade a Computação Quântica	Tempo para Quebra
RSA	Alta	Minutos a Horas
Diffie-Hellman	Alta	Minutos a Horas
ECC	Alta	Minutos a Horas
Criptografia Pós-Quântica	Baixa	Indefinido (resistente)

Tabela 3 – Comparação de Vulnerabilidade dos Algoritmos de Criptografia

para garantir a segurança das informações no futuro. A NIST está liderando esforços para identificar e padronizar algoritmos de criptografia pós-quântica, com o objetivo de fornecer uma base sólida para a segurança da informação na era da computação quântica. A preparação antecipada é essencial para mitigar os riscos associados à vulnerabilidade dos sistemas de chave pública atuais e garantir a proteção contínua dos dados sensíveis.

9.2 Famílias de Algoritmos Pós-Quânticos

9.2.1 Lattice-Based Cryptography

A criptografia baseada em reticulados é uma das famílias mais promissoras de algoritmos pós-quânticos. Ela se baseia em problemas matemáticos relacionados a reticulados, que são estruturas geométricas formadas por pontos em um espaço multidimensional. Esses algoritmos são considerados resistentes a ataques de computadores quânticos devido à dificuldade de resolver certos problemas relacionados a reticulados, como o problema do vetor mais curto (SVP) e o problema do vetor mais próximo (CVP). Exemplos de algoritmos de criptografia baseada em reticulados incluem NTRU e LWE (Learning With Errors). Esses algoritmos oferecem uma combinação de segurança e eficiência, tornando-os candidatos fortes para a criptografia pós-quântica.

9.2.2 Code-Based Cryptography

A criptografia baseada em códigos é outra família importante de algoritmos pós-quânticos. Ela se baseia em problemas relacionados a códigos de correção de erros, que são usados para detectar e corrigir erros em dados transmitidos. Algoritmos como McEliece e Niederreiter são exemplos de criptografia baseada em códigos. Esses algoritmos são considerados resistentes a ataques quânticos devido à dificuldade de decodificar certos tipos de códigos sem conhecer a chave secreta. Embora esses algoritmos possam ser menos eficientes em termos de tamanho da chave e desempenho, eles oferecem uma alternativa valiosa para a segurança pós-quântica.

9.2.3 Multivariate Cryptography

A criptografia multivariada é uma família de algoritmos pós-quânticos que se baseia em problemas relacionados a sistemas de equações polinomiais multivariadas. Esses algoritmos são considerados resistentes a ataques de computadores quânticos devido à dificuldade de resolver sistemas de equações polinomiais multivariadas sem conhecer a chave secreta. Exemplos de algoritmos de criptografia multivariada incluem Rainbow e HFE (Hidden Field Equations). Embora esses algoritmos possam ser menos eficientes em termos de desempenho, eles oferecem uma abordagem única para a criptografia pós-quântica e são candidatos valiosos para a segurança da informação no futuro.

9.2.4 Hash-Based Cryptography

A criptografia baseada em hash é uma família de algoritmos pós-quânticos que se baseia em funções de hash criptográficas. Esses algoritmos são considerados resistentes a ataques de computadores quânticos devido à dificuldade de inverter funções de hash sem conhecer a chave secreta. Exemplos de algoritmos de criptografia baseada em hash incluem Merkle Signature Scheme e XMSS (eXtended Merkle Signature Scheme). Embora esses algoritmos possam ser menos eficientes em termos de desempenho, eles oferecem uma abordagem robusta para a criptografia pós-quântica, especialmente para assinaturas digitais. A criptografia baseada em hash é uma opção valiosa para garantir a segurança das informações na era da computação quântica, proporcionando uma camada adicional de proteção contra ataques quânticos.

9.2.5 Isogeny-Based Cryptography

A criptografia baseada em isogenias é uma família de algoritmos pós-quânticos que se baseia em problemas relacionados a isogenias de curvas elípticas. Esses algoritmos são considerados resistentes a ataques de computadores quânticos devido à dificuldade de resolver certos problemas relacionados a isogenias sem conhecer a chave secreta. Exemplos de algoritmos de criptografia baseada em isogenias incluem SIDH (Supersingular Isogeny Diffie-Hellman) e SIKE (Supersingular Isogeny Key Encapsulation). Embora esses algoritmos possam ser menos eficientes em termos de desempenho, eles oferecem uma abordagem inovadora para a criptografia pós-quântica e são candidatos valiosos para a segurança da informação no futuro.

9.3 Estratégias de Transição e Agilidade Criptográfica

A transição para a criptografia pós-quântica é um processo complexo que requer uma abordagem estratégica e cuidadosa. Uma das estratégias mais importantes é a "agilidade

criptográfica", que se refere à capacidade de um sistema de segurança de se adaptar rapidamente a novas ameaças e tecnologias. Isso envolve a implementação de sistemas que possam suportar múltiplos algoritmos de criptografia, permitindo uma transição suave para algoritmos pós-quânticos à medida que eles se tornam disponíveis e padronizados. Além disso, é crucial que as organizações comecem a planejar e testar a implementação de algoritmos de criptografia pós-quântica em seus sistemas atuais para garantir uma transição eficiente e segura. A NIST está liderando esforços para desenvolver diretrizes e padrões para a transição para a criptografia pós-quântica, incluindo a identificação de algoritmos candidatos e a realização de testes de segurança. A preparação antecipada é essencial para mitigar os riscos associados à vulnerabilidade dos sistemas de chave pública atuais e garantir a proteção contínua dos dados sensíveis na era da computação quântica.

9.3.1 Definição de Agilidade Criptográfica (Cripto Agility)

A agilidade criptográfica, ou cripto agilidade, é a capacidade de um sistema de segurança de se adaptar rapidamente a novas ameaças e tecnologias. Isso envolve a implementação de sistemas que possam suportar múltiplos algoritmos de criptografia, permitindo uma transição suave para algoritmos pós-quânticos à medida que eles se tornam disponíveis e padronizados. A cripto agilidade é crucial para garantir a segurança das informações em um ambiente de ameaças em constante evolução, especialmente com o avanço da computação quântica. Ao adotar uma abordagem de cripto agilidade, as organizações podem mitigar os riscos associados à vulnerabilidade dos sistemas de chave pública atuais e garantir a proteção contínua dos dados sensíveis no futuro.

9.3.2 Insuficiência do Paradigma Bits-of-security

O paradigma "bits-of-security" é uma medida tradicional de segurança que quantifica a força de um algoritmo de criptografia com base no número de bits necessários para quebrá-lo. No entanto, esse paradigma é insuficiente para avaliar a segurança dos algoritmos pós-quânticos, pois não leva em consideração as ameaças específicas apresentadas pela computação quântica. A computação quântica pode quebrar algoritmos de chave pública tradicionais, como RSA e ECC, em um tempo relativamente curto, independentemente do número de bits de segurança. Portanto, é essencial adotar uma abordagem mais abrangente para avaliar a segurança dos algoritmos pós-quânticos, levando em consideração as vulnerabilidades específicas associadas à computação quântica e a necessidade de resistência a ataques quânticos. A NIST está trabalhando para desenvolver novos critérios de segurança que sejam mais adequados para avaliar os algoritmos de criptografia pós-quântica e garantir a proteção das informações na era da computação quântica.

9.4 O Plano de Padronização

O plano de padronização para a criptografia pós-quântica é um processo estruturado que visa identificar, avaliar e padronizar algoritmos de criptografia que sejam resistentes a ataques de computadores quânticos. A NIST está liderando esse esforço, realizando uma série de rodadas de avaliação para selecionar os algoritmos mais promissores para padronização. O processo envolve a submissão de algoritmos por parte da comunidade de pesquisa, seguida por uma avaliação rigorosa de sua segurança, eficiência e viabilidade prática. Os algoritmos selecionados passarão por um processo de padronização formal, que inclui a definição de especificações técnicas e diretrizes para sua implementação. O objetivo final é fornecer uma base sólida para a segurança da informação na era da computação quântica, garantindo que os dados sensíveis estejam protegidos contra ameaças futuras.

9.4.1 Processo Fora da Competição Tradicional

O processo de padronização para a criptografia pós-quântica da NIST é diferente do processo tradicional de competição, onde os algoritmos são avaliados e selecionados com base em critérios específicos. Em vez disso, a NIST adotou um processo mais colaborativo e iterativo, que envolve a participação ativa da comunidade de pesquisa em todas as etapas do processo. Isso inclui a submissão de algoritmos, a avaliação de segurança e eficiência, e a definição de especificações técnicas. O objetivo é criar um ambiente de colaboração que permita a identificação e desenvolvimento dos melhores algoritmos de criptografia pós-quântica, garantindo que eles sejam adequados para uso prático e resistentes a ataques quânticos. Esse processo fora da competição tradicional é essencial para garantir que os algoritmos selecionados sejam robustos, eficientes e capazes de proteger as informações na era da computação quântica.

9.4.2 Critérios Iniciais e Prazos

Os critérios iniciais para a avaliação dos algoritmos de criptografia pós-quântica incluem segurança, eficiência, viabilidade prática e resistência a ataques quânticos. A NIST estabeleceu prazos para cada etapa do processo de padronização, com a expectativa de selecionar os algoritmos finalistas em um período de 3 a 5 anos. Durante esse tempo, os algoritmos serão submetidos a uma avaliação rigorosa por parte da comunidade de pesquisa, que incluirá testes de segurança e desempenho. A NIST também planeja realizar rodadas adicionais de avaliação para garantir que os algoritmos selecionados sejam robustos e adequados para uso prático. O objetivo é garantir que os algoritmos padronizados sejam capazes de proteger as informações contra ameaças futuras, especialmente aquelas apresentadas pela computação quântica.

10 NIST IR 8547 Transição para a Criptografia Pós-Quântica: Estratégias e Considerações

10.1 Urgencia Prática e o Modelo de Ameaça

O modelo de ameaça é uma ferramenta essencial para a segurança cibernética, permitindo que as organizações identifiquem, compreendam e mitiguem ameaças potenciais. Ele ajuda a priorizar os recursos de segurança, focando nas ameaças mais relevantes e perigosas. O modelo de ameaça é fundamental para o desenvolvimento de estratégias eficazes de defesa cibernética, garantindo que as organizações estejam preparadas para enfrentar os desafios atuais e futuros.

10.1.1 Ameaça Harvest now, Decrypt Later

A ameaça "Harvest now, Decrypt Later" refere-se à prática de coletar dados criptografados agora, com a intenção de descriptografá-los no futuro, quando os recursos computacionais ou as vulnerabilidades de segurança permitirem. Essa ameaça é particularmente preocupante para dados sensíveis que precisam ser protegidos por longos períodos, como informações pessoais, financeiras e de saúde. Para mitigar essa ameaça, é crucial adotar práticas de criptografia robustas e atualizadas, além de monitorar continuamente as vulnerabilidades e avanços tecnológicos que possam impactar a segurança dos dados.

10.1.2 Teorema de Mosca e a Linha do Tempo

O Teorema de Mosca é um conceito fundamental na segurança cibernética que destaca a importância de considerar o tempo ao avaliar ameaças e vulnerabilidades. Ele sugere que, para proteger adequadamente os sistemas, é necessário entender não apenas as ameaças atuais, mas também as ameaças futuras que podem surgir à medida que a tecnologia evolui. A linha do tempo é uma ferramenta útil para visualizar e planejar a defesa contra ameaças, permitindo que as organizações antecipem e se preparem para possíveis ataques futuros, garantindo uma postura de segurança proativa.

10.2 Novos Padrões e Categorias de Segurança

A evolução dos padrões de segurança é crucial para enfrentar as ameaças cibernéticas em constante mudança. Novos padrões e categorias de segurança são desenvolvidos para abordar as vulnerabilidades emergentes e garantir a proteção eficaz dos sistemas e dados. A adoção de padrões atualizados é essencial para manter a resiliência contra ataques

cibernéticos, promovendo uma abordagem de segurança mais robusta e adaptável às novas ameaças.

10.2.1 Publicações dos primeiros FIPS PQC

As publicações dos primeiros FIPS (Federal Information Processing Standards) para criptografia pós-quântica (PQC) representam um marco importante na evolução da segurança cibernética. Esses padrões são desenvolvidos para garantir que os sistemas de criptografia sejam resistentes a ataques de computadores quânticos, que têm o potencial de quebrar os métodos de criptografia tradicionais. A adoção desses padrões é crucial para proteger dados sensíveis e garantir a segurança a longo prazo, especialmente à medida que a tecnologia quântica avança e se torna mais acessível.

10.2.2 Categorias de Segurança Pós-Quântica (1 a 5)

As categorias de segurança pós-quântica são classificações que ajudam a identificar e categorizar os diferentes tipos de algoritmos de criptografia que são resistentes a ataques de computadores quânticos. Essas categorias incluem algoritmos baseados em reticulados, códigos, multivariados, hash-based e isogenias, cada um com suas próprias características e níveis de segurança. A compreensão dessas categorias é essencial para escolher os algoritmos mais adequados para proteger dados sensíveis contra ameaças futuras, garantindo uma postura de segurança robusta e resiliente.

10.3 Cronograma de Descontinuação (Deprecation)

O cronograma de descontinuação é um plano estratégico que define o processo e os prazos para a retirada de sistemas, tecnologias ou práticas de segurança obsoletas. Ele é essencial para garantir uma transição suave e segura para novas soluções, minimizando os riscos associados à manutenção de sistemas vulneráveis ou desatualizados. A implementação de um cronograma de descontinuação eficaz ajuda as organizações a se adaptarem às mudanças tecnológicas e a manterem uma postura de segurança proativa, garantindo a proteção contínua dos dados e sistemas.

10.3.1 Transição de Assinaturas Digitais (RSA, ECDSA, EdDSA)

A transição de assinaturas digitais, como RSA, ECDSA e EdDSA, é um processo crucial para garantir a segurança dos sistemas de criptografia à medida que novas ameaças emergem. Com o avanço da tecnologia, especialmente a computação quântica, é necessário migrar para algoritmos de assinatura digital mais robustos e resistentes a ataques futuros. A adoção de novos algoritmos de assinatura digital é essencial para proteger a integridade e

autenticidade dos dados, garantindo a segurança a longo prazo em um ambiente cibernético em constante evolução.

10.3.2 Transição de Estabelecimento de Chaves (DH, MQV, RSA)

A transição de estabelecimento de chaves, como DH (Diffie-Hellman), MQV (Menezes-Qu-Vanstone) e RSA, é um processo fundamental para garantir a segurança das comunicações criptografadas. Com o avanço da tecnologia, especialmente a computação quântica, é necessário migrar para algoritmos de estabelecimento de chaves mais robustos e resistentes a ataques futuros. A adoção de novos algoritmos de estabelecimento de chaves é essencial para proteger a confidencialidade e integridade das comunicações, garantindo a segurança a longo prazo em um ambiente cibernético em constante evolução.

10.4 Estratégias de Transição: Protocolos Híbridos

As estratégias de transição, como os protocolos híbridos, são abordagens que combinam algoritmos de criptografia tradicionais com algoritmos pós-quânticos para garantir a segurança durante o processo de migração. Esses protocolos permitem que as organizações mantenham a compatibilidade com sistemas legados enquanto adotam novas soluções de segurança, minimizando os riscos associados à transição. A implementação de protocolos híbridos é uma estratégia eficaz para garantir a proteção contínua dos dados e sistemas durante a evolução da tecnologia, proporcionando uma transição suave e segura para o futuro da segurança cibernética.

10.4.1 Soluções Híbridas (PQC-Classical)

As soluções híbridas, que combinam algoritmos de criptografia pós-quântica (PQC) com algoritmos clássicos, são uma abordagem estratégica para garantir a segurança durante a transição para novas tecnologias de criptografia. Essas soluções permitem que as organizações mantenham a compatibilidade com sistemas legados enquanto adotam novas soluções de segurança, minimizando os riscos associados à migração. A implementação de soluções híbridas é essencial para proteger os dados e sistemas durante a evolução da tecnologia, garantindo uma transição suave e segura para o futuro da segurança cibernética.

10.4.2 Protocolos Híbridos (TLS, SSH, IPsec)

Os protocolos híbridos, como TLS (Transport Layer Security), SSH (Secure Shell) e IPsec (Internet Protocol Security), são abordagens que combinam algoritmos de criptografia tradicionais com algoritmos pós-quânticos para garantir a segurança durante a transição para novas tecnologias de criptografia. Esses protocolos permitem que as organizações

mantenham a compatibilidade com sistemas legados enquanto adotam novas soluções de segurança, minimizando os riscos associados à migração. A implementação de protocolos híbridos é essencial para proteger os dados e sistemas durante a evolução da tecnologia, garantindo uma transição suave e segura para o futuro da segurança cibernética.

10.4.3 Agilidade Criptográfica em Camadas de Infraestrutura

A agilidade criptográfica em camadas de infraestrutura é uma abordagem que permite a flexibilidade na escolha e implementação de algoritmos de criptografia em diferentes camadas de um sistema. Essa abordagem é crucial para garantir a segurança durante a transição para novas tecnologias de criptografia, permitindo que as organizações adotem soluções híbridas e se adaptem às mudanças nas ameaças cibernéticas. A implementação da agilidade criptográfica em camadas de infraestrutura é essencial para proteger os dados e sistemas durante a evolução da tecnologia, garantindo uma transição suave e segura para o futuro da segurança cibernética.

11 Cristals Kyber

11.1 Introdução

O CRYSTALS-Kyber é um sistema de criptografia de chave pública baseado em problemas de aprendizado com erros (LWE) e é considerado resistente a ataques de computadores quânticos. Ele é parte do projeto CRYSTALS (Cryptographic Suite for Algebraic Lattices) e é um dos candidatos para o padrão de criptografia pós-quântica do NIST (National Institute of Standards and Technology). O CRYSTALS-Kyber é projetado para ser eficiente em termos de tempo de execução e tamanho de chave, tornando-o uma opção viável para aplicações práticas de criptografia pós-quântica.

11.2 Estrutura do CRYSTALS-Kyber

O CRYSTALS-Kyber é baseado em um esquema de criptografia de chave pública que utiliza matrizes e vetores para realizar operações de criptografia e descriptografia. Ele é composto por três algoritmos principais: geração de chaves, criptografia e descriptografia.

12 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)

12.1 Definição e Objetivos da PQC

Algoritmos clássicos projetados para resistir a ataques quânticos.

12.2 Paradigmas Matemáticos da PQC

Reticulados, códigos corretores de erro, funções hash e isogenias.

12.3 Criptografia Baseada em Reticulados

Introdução ao conceito de Lattices como base da nova segurança criptográfica.

12.4 Learning With Errors (LWE)

Fundamento matemático da pseudoaleatoriedade pós-quântica.

12.4.1 Implementação Didática em Java (Demonstração)

Implementação simplificada do LWE do zero em Java para fins didáticos, mostrando como o erro intencional funciona matematicamente antes de utilizar a biblioteca pronta.

12.5 Padronização Pós-Quântica

O papel do ML-KEM (Kyber) e o processo de padronização conduzido pelo NIST.

13 A TRANSIÇÃO PARA PQC NO ECOSISTEMA JAVA

13.1 Suporte à Criptografia Pós-Quântica no Java

Evolução do JDK e integração de algoritmos pós-quânticos em versões modernas (Java 21+).

13.2 Crypto-Agility

Boas práticas para desenvolvimento de sistemas criptográficos flexíveis e preparados para mudanças de algoritmo.

13.2.1 Implementação Prática: Demo de Crypto-Agility

Criação de uma interface Java que abstrai o algoritmo de geração de chaves, permitindo trocar de RSA para Kyber alterando apenas a configuração — sem reescrever o código da aplicação.

13.3 Comparativo de Modelos

Geração de chaves e aleatoriedade no modelo clássico versus modelo baseado em reticulados (Module-Lattice).

13.3.1 Implementação Prática: Benchmark Clássico vs. Kyber

Benchmark comparativo entre `java.security.SecureRandom` clássico e ML-KEM (Kyber) via Bouncy Castle, medindo tempo de geração de chaves, uso de memória e throughput. Inclui comparativo de tamanho de chaves entre RSA-2048 e Kyber. Nota: Confirmar com o orientador o uso da biblioteca Bouncy Castle como dependência, por ser a opção mais acessível para PQC em Java atualmente.

14 DISCUSSÃO E ANÁLISE CRÍTICA

14.1 Impacto em Desempenho

Custos computacionais, consumo de memória e latência — baseados nos resultados do benchmark da seção 6.3.1.

14.2 Barreiras de Adoção

Sistemas legados, limitações de hardware e resistência organizacional.

14.3 Implicações Sociais e Éticas

Soberania de dados, privacidade, vigilância e assimetria tecnológica.

15 CONCLUSÃO

15.1 Síntese dos Resultados

A inevitabilidade da transição criptográfica.

15.2 Resposta ao Problema de Pesquisa

A evolução da pseudoaleatoriedade como pilar da segurança pós-quântica.

15.3 Trabalhos Futuros

Geradores Quânticos de Números Aleatórios (QRNG) e integração híbrida TRNG + PQC.

Referências